The Knapsack Problem

Branch and Bound and Dynamic Programming
Optimization Methods, Week 9

DSBA

The 0–1 knapsack problem

Definition (0–1 knapsack)

For positive item profits p_i, positive weights w_i, and capacity W,

$$\max_{x \in \{0,1\}^n} \sum_{i=1}^{n} p_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq W.$$

The mathematical problem permits real weights. The capacity-indexed DP later in this lecture requires integer weights and integer capacity.

A source example

$$p = (10, 10, 12, 18), \qquad w = (2, 4, 6, 9), \qquad W = 15.$$

The selection $(1, 1, 0, 1)$ weighs 15 and earns 38. The selection $(1, 1, 1, 0)$ weighs 12 and earns 32. The task is to prove that no other feasible subset earns more than 38, without checking all 2^4 vectors by hand.

Variations from the source lecture

Model	Decision variables	Additional structure
Bounded	$x_i \in \{0, \dots, b_i\}$	limited identical copies
Unbounded	$x_i \in \mathbb{Z}_{\geq 0}$	unlimited copies
Multidimensional	$x_i \in \{0, 1\}$	several capacities
Multiple knapsack	$x_{ij} \in \{0, 1\}$	several containers

The subset-sum special case sets $p_i = w_i$ and asks whether the optimum equals W.

Bounded, unbounded, and multidimensional models

Bounded knapsack uses $0 \le x_i \le b_i$, $x_i \in \mathbb{Z}$. Unbounded knapsack uses $x_i \in \mathbb{Z}_{\ge 0}$; positive weights keep the number of feasible copies finite. With d resource constraints,

$$\sum_i w_{ki} x_i \le W_k, \qquad k = 1, \dots, d.$$

For integer capacities, a direct DP can require roughly $\prod_{k=1}^{d}(W_k + 1)$ states per item. The state space grows rapidly with d.

Several knapsacks

Let $x_{ij} = 1$ if item i enters knapsack j. Then

$$\max \sum_i \sum_j p_i x_{ij}, \qquad \sum_i w_i x_{ij} \le W_j, \qquad \sum_j x_{ij} \le 1, \qquad x_{ij} \in \{0,1\}.$$

An item goes into at most one knapsack. Replacing the separate W_j by their sum can admit assignments that do not fit any single container.

Why exhaustive search is expensive

There are 2^n binary vectors. At $n = 50$, this exceeds 10^{15}.

- Branch and bound prunes a set of vectors when a valid upper bound cannot beat the best known feasible value.
- Dynamic programming evaluates an item-and-capacity state once and reuses it.

Neither method is guaranteed polynomial time in the length of a binary-encoded input.

Branch-and-bound nodes

A node fixes some item decisions. Store fixed profit P, fixed weight C, and remaining capacity $R = W - C$. Branch on an unfixed item j:

$$x_j = 0 \quad \text{or} \quad x_j = 1.$$

The second child is infeasible if $w_j > R$. A feasible selection found anywhere in the tree provides an incumbent value z_{inc}.

Upper bounds and safe pruning

Definition (Valid upper bound)

For a node N, $U(N)$ is valid if every feasible completion x in N satisfies $f(x) \leq U(N)$.

Theorem (Pruning rule)

If $U(N) \leq z_{\text{inc}}$, node N cannot contain a better solution.

Upper bounds and safe pruning

Definition (Valid upper bound)

For a node N, $U(N)$ is valid if every feasible completion x in N satisfies $f(x) \leq U(N)$.

Theorem (Pruning rule)

If $U(N) \leq z_{\mathrm{inc}}$, node N cannot contain a better solution.

Take any feasible completion x in N.

Upper bounds and safe pruning

Definition (Valid upper bound)

For a node N, $U(N)$ is valid if every feasible completion x in N satisfies $f(x) \leq U(N)$.

Theorem (Pruning rule)

If $U(N) \leq z_{\mathrm{inc}}$, node N cannot contain a better solution.

Take any feasible completion x in N. Validity gives $f(x) \leq U(N)$.

Upper bounds and safe pruning

Definition (Valid upper bound)

For a node N, $U(N)$ is valid if every feasible completion x in N satisfies $f(x) \leq U(N)$.

Theorem (Pruning rule)

If $U(N) \leq z_{\mathrm{inc}}$, node N cannot contain a better solution.

Take any feasible completion x in N. Validity gives $f(x) \leq U(N)$. The pruning test gives $U(N) \leq z_{\mathrm{inc}}$.

Upper bounds and safe pruning

Definition (Valid upper bound)

For a node N, $U(N)$ is valid if every feasible completion x in N satisfies $f(x) \leq U(N)$.

Theorem (Pruning rule)

If $U(N) \leq z_{\mathrm{inc}}$, node N cannot contain a better solution.

Take any feasible completion x in N. Validity gives $f(x) \leq U(N)$. The pruning test gives $U(N) \leq z_{\mathrm{inc}}$. Therefore $f(x) \leq z_{\mathrm{inc}}$. Discarding N is safe. □

A fractional relaxation gives an upper bound

At a node, let S be the remaining items. Relax their binary variables:

$$U(N) = P + \max \left\{ \sum_{i \in S} p_i x_i : \sum_{i \in S} w_i x_i \le R, \quad 0 \le x_i \le 1 \right\}.$$

Every feasible integer completion belongs to this larger feasible set. Its relaxed optimum therefore bounds the integer optimum from above.

Why density sorting solves the relaxation

Theorem (Fractional knapsack)

Sorting by decreasing p_i/w_i, taking full items and at most one fractional item, solves the relaxation.

Why density sorting solves the relaxation

Theorem (Fractional knapsack)

Sorting by decreasing p_i/w_i, taking full items and at most one fractional item, solves the relaxation.

Suppose a lower-density item j uses positive weight while a higher-density item i is not full.

Why density sorting solves the relaxation

Theorem (Fractional knapsack)

Sorting by decreasing p_i/w_i, taking full items and at most one fractional item, solves the relaxation.

Suppose a lower-density item j uses positive weight while a higher-density item i is not full. Move a small weight δ from j to i, preserving feasibility.

Why density sorting solves the relaxation

Theorem (Fractional knapsack)

Sorting by decreasing p_i/w_i, taking full items and at most one fractional item, solves the relaxation.

Suppose a lower-density item j uses positive weight while a higher-density item i is not full. Move a small weight δ from j to i, preserving feasibility. The profit change is $\delta(p_i/w_i - p_j/w_j) \geq 0$.

Why density sorting solves the relaxation

Theorem (Fractional knapsack)

Sorting by decreasing p_i/w_i, taking full items and at most one fractional item, solves the relaxation.

Suppose a lower-density item j uses positive weight while a higher-density item i is not full. Move a small weight δ from j to i, preserving feasibility. The profit change is $\delta(p_i/w_i - p_j/w_j) \geq 0$. Repeat until a density-sorted prefix is full and at most one further item is fractional. □

Calculating the fractional bound

After sorting the remaining items, fill the capacity R. Suppose the first item that does not fit is t, and R' remains. Then

$$U(N) = P + \sum_{\text{full } i} p_i + \frac{R'}{w_t} p_t.$$

The fractional last item is generally not a feasible binary choice. A separate feasible greedy fill can improve the incumbent.

A branch-and-bound trace

Let $p = (60, 100, 120)$, $w = (10, 20, 30)$, $W = 50$.

Node	Fixed choice	Upper bound	Feasible value
Root	none	240	160 from $(1, 1, 0)$
Left	$x_1 = 0$	220	220 from $(0, 1, 1)$
Right	$x_1 = 1$	240	branch further

After the left child, the incumbent is 220. Every remaining node with bound at most 220 can be pruned.

Branch-and-bound workflow

1. Initialize a feasible incumbent and place the root in a queue.
2. Select a node and calculate a valid upper bound.
3. Prune if infeasible or if its bound is no better than the incumbent.
4. Otherwise branch on an unfixed variable and inspect both children.
5. Update the incumbent whenever a better feasible completion appears.

Depth-first search uses little memory. Best-bound search often discovers a strong bound early but can hold many live nodes.

Why complete branch and bound is exact

Theorem (Finite exact search)

For a finite 0–1 instance, valid bounds and complete branching give an optimal incumbent when the queue is empty.

Why complete branch and bound is exact

Theorem (Finite exact search)

For a finite 0–1 instance, valid bounds and complete branching give an optimal incumbent when the queue is empty.

Every branch fixes another binary variable, so the search tree has finite depth at most n.

Why complete branch and bound is exact

Theorem (Finite exact search)

For a finite 0–1 instance, valid bounds and complete branching give an optimal incumbent when the queue is empty.

Every branch fixes another binary variable, so the search tree has finite depth at most n. A pruned node cannot improve the incumbent by the pruning theorem.

Why complete branch and bound is exact

Theorem (Finite exact search)

For a finite 0–1 instance, valid bounds and complete branching give an optimal incumbent when the queue is empty.

Every branch fixes another binary variable, so the search tree has finite depth at most n. A pruned node cannot improve the incumbent by the pruning theorem. Every unpruned leaf is infeasible or is compared with the incumbent.

Why complete branch and bound is exact

Theorem (Finite exact search)

For a finite 0–1 instance, valid bounds and complete branching give an optimal incumbent when the queue is empty.

Every branch fixes another binary variable, so the search tree has finite depth at most n. A pruned node cannot improve the incumbent by the pruning theorem. Every unpruned leaf is infeasible or is compared with the incumbent. No feasible vector can exceed the final incumbent, and the incumbent is feasible. □

Limits of branch and bound

A weak upper bound may leave nearly all 2^n branches alive. Runtime depends on bound quality, branch order, and how quickly the method finds a strong incumbent. An unfinished run can still report a certificate: the incumbent is a lower bound on the optimum, while the largest bound over live nodes is an upper bound.

Integer weights are required for this DP

Assumption for capacity-indexed DP

$w_i \in \mathbb{Z}_{>0}$ and $W \in \mathbb{Z}_{\geq 0}$. The table has columns for capacities $0, 1, \ldots, W$.

Rational weights can be scaled to integers exactly, but the new capacity may be very large. Arbitrary real weights do not give this finite integer-capacity table directly.

DP state and boundary values

Definition (State)

$D(i, c)$ is the best profit using the first i items with integer capacity c.

$$D(0, c) = 0 \quad (0 \le c \le W), \qquad D(i, 0) = 0$$

because every weight is positive. The answer is $D(n, W)$.

The 0–1 recurrence

$$D(i,c) = \begin{cases} D(i-1,c), & w_i > c, \\ \max\{D(i-1,c),\ p_i + D(i-1,c-w_i)\}, & w_i \le c. \end{cases}$$

Both alternatives use row $i-1$, ensuring that item i appears at most once.

Proof of the recurrence

Theorem (Optimal substructure)

The recurrence computes the optimal value for every state (i, c).

Proof of the recurrence

Theorem (Optimal substructure)

The recurrence computes the optimal value for every state (i, c).

An optimal subset of the first i items either excludes item i or includes it.

Proof of the recurrence

Theorem (Optimal substructure)

The recurrence computes the optimal value for every state (i, c).

An optimal subset of the first i items either excludes item i or includes it. If it excludes i, its best possible profit is $D(i-1, c)$.

Proof of the recurrence

Theorem (Optimal substructure)

The recurrence computes the optimal value for every state (i, c).

An optimal subset of the first i items either excludes item i or includes it. If it excludes i, its best possible profit is $D(i-1, c)$. If it includes i, then $w_i \leq c$, and its best possible profit is $p_i + D(i-1, c-w_i)$.

Proof of the recurrence

Theorem (Optimal substructure)

The recurrence computes the optimal value for every state (i, c).

An optimal subset of the first i items either excludes item i or includes it. If it excludes i, its best possible profit is $D(i-1, c)$. If it includes i, then $w_i \leq c$, and its best possible profit is $p_i + D(i-1, c-w_i)$. Both cases are achievable. Their maximum is optimal. Induction from row 0 proves the table. □

A compact table for the source example

$$p = (10, 10, 12, 18), \quad w = (2, 4, 6, 9), \quad W = 15.$$

$i \backslash c$	0	2	4	6	9	12	15
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	10	20	20	20	20
3	0	10	10	20	22	32	32
4	0	10	10	20	22	32	38

Only selected columns are shown. The algorithm fills every integer capacity. Here $D(4, 15) = 38$.

Reconstructing the selection

Start at $(i, c) = (n, W)$. If $D(i, c) = D(i - 1, c)$, exclude item i. Otherwise include it and set $c \leftarrow c - w_i$. Decrease i at each step. For the example, take item 4 and move from capacity 15 to 6. Then take item 2 and move to 2. Finally take item 1. The solution is $(1, 1, 0, 1)$.

Runtime and memory

$$\text{time } O(nW), \qquad \text{full table memory } O(nW).$$

A one-row implementation needs $O(W)$ memory for the value only. To update a 0–1 item in one row, scan capacities from W downward to w_i. This preserves the previous row values at $c - w_i$. An ascending scan would permit repeated use of item i, solving a different recurrence.

DP for bounded and unbounded copies

Bounded copies $0 \le x_i \le b_i$ give

$$D(i,c) = \max_{0 \le q \le \min(b_i, \lfloor c/w_i \rfloor)} \{qp_i + D(i-1, c-qw_i)\}.$$

For unlimited copies,

$$D(i,c) = \max\{D(i-1,c),\ p_i + D(i,c-w_i)\},$$

when $w_i \le c$. The second term stays in row i, allowing item i again.

Pseudo-polynomial time

The capacity W is written with approximately $\log_2 W$ bits. If $W = 2^k$, a table with $W + 1$ columns is exponential in k, despite the $O(nW)$ running time.
Capacity-indexed DP is effective when integer W is moderate. It becomes impractical when capacities or exact scaling factors are huge.

Subset sum reduction

Theorem

Subset sum reduces to deciding whether a 0–1 knapsack optimum equals its capacity.

This explains how a pseudo-polynomial algorithm can coexist with NP-hardness.

Subset sum reduction

Theorem

Subset sum reduces to deciding whether a 0–1 knapsack optimum equals its capacity.

Given positive integer weights w_i and target W, set $p_i = w_i$.

This explains how a pseudo-polynomial algorithm can coexist with NP-hardness.

Subset sum reduction

Theorem

Subset sum reduces to deciding whether a 0–1 knapsack optimum equals its capacity.

Given positive integer weights w_i and target W, set $p_i = w_i$. Every feasible subset has profit equal to its weight and no more than W.

This explains how a pseudo-polynomial algorithm can coexist with NP-hardness.

Subset sum reduction

Theorem

Subset sum reduces to deciding whether a 0–1 knapsack optimum equals its capacity.

Given positive integer weights w_i and target W, set $p_i = w_i$. Every feasible subset has profit equal to its weight and no more than W. The optimum equals W exactly when a subset sums to W. $\square$ This explains how a pseudo-polynomial algorithm can coexist with NP-hardness.

What if weights are not integers?

The capacity-indexed DP shown here requires integer weights and capacity. Exact scaling works for rational inputs but can create a huge table. Rounding weights changes the instance and may change its optimum. Alternative DP formulations exist:

profit-indexed DP can use integer profits and store minimum required weight. Branch and bound can work directly with positive real weights.

Choosing an exact method

	Branch and bound	Capacity-indexed DP
Key resource	surviving search nodes	integer capacity W
Weights	positive real or integer	positive integers after exact scaling
Best setting	strong bounds, early incumbent	moderate W, repeated states
Worst case	exponential tree	pseudo-polynomial $O(nW)$

Takeaways

- Knapsack variants change copy limits, resources, or assignments to containers.
- Branch and bound is exact because a pruned node has a valid upper bound no better than the incumbent.
- Capacity-indexed DP is exact for positive integer weights and integer capacity. Its cost depends on the numeric value of W.
- Traceback subtracts a chosen item's weight from the current capacity.

Adapted from A. D. Ignatov, *Optimization Methods*, Lecture 10 (2025–2026). Further reading: Kellerer, Pferschy, and Pisinger, *Knapsack Problems*; Cormen et al., *Introduction to Algorithms*.